PROTOCOL: ZERO-TOUCH TIMELINE CORE: GEMINI-LIVE-3.1 STATUS: FULL AUTONOMOUS

I Gave JARVIS Hands-Free Control Over YouTube! (EP 13) ■■

ARCHITECT: SWAYAM DHAWALE (PULSE OF AI) // STARK OS MASTER BLUEPRINT

■ [CRITICAL INITIALIZATION DIRECTIVE // PREREQUISITE ALERT]

SYSTEM NOTICE: The JARVIS core operates on a modular pipeline architecture. Each consecutive update inherits system hooks, thread optimizers, hardware audio pipelines, and GSMTC inspectors. **YOU MUST WATCH ALL PRECEDING EPISODES (1 TO 12) AND EXTRACT REQUISITE CODE MODULES FROM THEIR PINNED COMMENTS.** Running Episode 13 in isolation without historical foundation modules will cause severe dependency breaks and boot failure.

■ ARCHIVAL INDEX // HISTORICAL SYSTEM EVOLUTION (EPISODES 01–12)

NODE	SUBSYSTEM PROTOCOL IMPLEMENTED	HUD TERMINAL TELEMETRY LINK
EP 01	Genesis & Voice Core Bridge	youtube.com/shorts/4fvNmeZ0E68?feature=share
EP 02	Neural Bridge & Low-Latency TTS	youtube.com/shorts/s2sz62HNmbE?feature=share
EP 03	Bilingual Language Routing Matrix	youtube.com/shorts/qhaxKI-aocU?feature=share
EP 04	OS System Process Injectors	youtube.com/shorts/WvV8ss0jn2o?feature=share
EP 05	Deep Task Optimizer & Thread Purge	youtube.com/shorts/7DCTFvycGe4?feature=share
EP 06	Hardware Telemetry Thermal Sensors	youtube.com/shorts/ek-iZBw-vjY?feature=share
EP 07	Zero-Touch Full Desktop Command	youtube.com/shorts/6kbS4zYSN2Q?feature=share
EP 08	Dynamic Adaptive Audio & VAD Gate	youtube.com/shorts/FWWldND5gQ4?feature=share
EP 09	Universal UWP Application Router	youtube.com/shorts/9ATHesGlVdA?feature=share
EP 10	Ultra-Low-Latency Dual Streaming	youtube.com/shorts/g8v89plnk-Y?feature=share
EP 11	Active Media Audio Ducking Layer	youtube.com/shorts/FJvn6oonfk8?feature=share
EP 12	Active GSMTC Media Bus Inspector	youtube.com/shorts/nbP_oir2YKc?feature=share

■ TERMINAL I // software_control.py (OS Key Injector & Media Bus)

```
"""
software_control.py
Universal Application, Deep Task Purge & Full Media/Browser Navigator for JARVIS (Episode 13)
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import os
import sys
import ctypes
import subprocess
import webbrowser
import asyncio
import psutil
import pyautogui

try:
    import win32gui
    import win32con
    import win32process
except ImportError:
    win32gui = None

try:
    from winsdk.windows.media.control import (
        GlobalSystemMediaTransportControlsSessionManager as MediaManager
    )
except ImportError:
    MediaManager = None

VK_MEDIA_NEXT_TRACK = 0xB0
VK_MEDIA_PREV_TRACK = 0xB1
VK_MEDIA_STOP       = 0xB2
VK_MEDIA_PLAY_PAUSE = 0xB3
VK_VOLUME_MUTE      = 0xAD
VK_K                = 0x4B
VK_DOWN             = 0x28
VK_UP               = 0x26
VK_LEFT             = 0x25
VK_RIGHT            = 0x27
VK_J                = 0x4A
VK_L                = 0x4C
VK_F                = 0x46
VK_T                = 0x54
KEYEVENTF_EXTENDEDKEY = 0x0001
KEYEVENTF_KEYUP       = 0x0002

def _send_hardware_media_key(key_code: int):
    ctypes.windll.user32.keybd_event(key_code, 0, KEYEVENTF_EXTENDEDKEY, 0)
    ctypes.windll.user32.keybd_event(key_code, 0, KEYEVENTF_EXTENDEDKEY | KEYEVENTF_KEYUP, 0)

def _find_browser_windows():
    browser_windows = []
    target_exes = ["chrome.exe", "msedge.exe", "brave.exe", "firefox.exe"]
    if win32gui is None:
        return browser_windows

    def enum_windows_callback(hwnd, extra):
        if win32gui.IsWindowVisible(hwnd):
            title = win32gui.GetWindowText(hwnd).strip()
            if title:
                try:
                    _, pid = win32process.GetWindowThreadProcessId(hwnd)
                    p = psutil.Process(pid)
                    if p.name().lower() in target_exes:
```

```
                        browser_windows.append((hwnd, p.name().lower(), title))
                except Exception:
                    pass
        return True

    win32gui.EnumWindows(enum_windows_callback, None)
    return browser_windows

def _send_key_to_browser(key_code: int) -> bool:
    windows = _find_browser_windows()
    if not windows or win32gui is None:
        return False
    target_hwnd = None
    for hwnd, name, title in windows:
        if "youtube" in title.lower() or "shorts" in title.lower():
            target_hwnd = hwnd
            break
    if not target_hwnd and windows:
        target_hwnd = windows[0][0]
    if target_hwnd:
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYDOWN, key_code, 0)
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYUP, key_code, 0)
        return True
    return False

def _send_browser_play_pause_background() -> bool:
    windows = _find_browser_windows()
    if not windows or win32gui is None:
        return False
    target_hwnd = None
    for hwnd, name, title in windows:
        if "youtube" in title.lower() or "shorts" in title.lower():
            target_hwnd = hwnd
            break
    if not target_hwnd and windows:
        target_hwnd = windows[0][0]
    if target_hwnd:
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYDOWN, VK_K, 0)
        win32gui.PostMessage(target_hwnd, win32con.WM_KEYUP, VK_K, 0)
        win32gui.PostMessage(target_hwnd, win32con.WM_APPCOMMAND, 0, 0xE0000)
        return True
    return False

async def _check_media_playing_async() -> bool:
    if MediaManager is None:
        return False
    try:
        manager = await MediaManager.request_async()
        session = manager.get_current_session()
        if not session:
            sessions = manager.get_sessions()
            if sessions and len(sessions) > 0:
                session = sessions[0]
        if session:
            info = session.get_playback_info()
            if info:
                return int(info.playback_status) == 4
    except Exception:
        pass
    return False

def is_media_actively_playing() -> bool:
    try:
        loop = asyncio.new_event_loop()
```

```
        asyncio.set_event_loop(loop)
        result = loop.run_until_complete(_check_media_playing_async())
        loop.close()
        return bool(result)
    except Exception:
        return False

def navigate_shorts(direction: str = "next") -> str:
    cmd = direction.lower().strip()
    if cmd in ["next", "down", "forward"]:
        if _send_key_to_browser(VK_DOWN):
            return "Skipped to next Short, sir."
        pyautogui.press("down")
        return "Scrolled to next Short, sir."
    elif cmd in ["previous", "prev", "up", "back"]:
        if _send_key_to_browser(VK_UP):
            return "Returned to previous Short, sir."
        pyautogui.press("up")
        return "Scrolled to previous Short, sir."
    return "Direction must be 'next' or 'previous', sir."

def skip_video(direction: str = "next") -> str:
    cmd = direction.lower().strip()
    if cmd in ["next", "forward", "skip"]:
        pyautogui.hotkey("shift", "n")
        return "Playing next video, sir."
    elif cmd in ["previous", "prev", "back"]:
        pyautogui.hotkey("shift", "p")
        return "Playing previous video, sir."
    return "Invalid direction, sir."

def seek_video(action: str = "forward") -> str:
    cmd = action.lower().strip()
    if cmd in ["forward", "fast_forward", "ahead"]:
        if _send_key_to_browser(VK_L):
            return "Advanced 10 seconds, sir."
        pyautogui.press("l")
        return "Skipped 10 seconds forward, sir."
    elif cmd in ["rewind", "back", "backward"]:
        if _send_key_to_browser(VK_J):
            return "Rewound 10 seconds, sir."
        pyautogui.press("j")
        return "Rewound 10 seconds, sir."
    return "Action must be 'forward' or 'rewind', sir."

def toggle_video_view(mode: str = "fullscreen") -> str:
    cmd = mode.lower().strip()
    if cmd in ["fullscreen", "full_screen", "full"]:
        if _send_key_to_browser(VK_F):
            return "Fullscreen toggled, sir."
        pyautogui.press("f")
        return "Fullscreen mode toggled, sir."
    elif cmd in ["theater", "theatre", "cinema"]:
        if _send_key_to_browser(VK_T):
            return "Theater mode toggled, sir."
        pyautogui.press("t")
        return "Theater mode toggled, sir."
    return "Mode must be 'fullscreen' or 'theater', sir."

async def control_media(action: str) -> str:
    cmd = action.lower().strip().replace(" ", "_")
    session = None
    if MediaManager is not None:
        try:
```

```
            manager = await MediaManager.request_async()
            session = manager.get_current_session()
            if not session:
                sessions = manager.get_sessions()
                if sessions and len(sessions) > 0:
                    session = sessions[0]
        except Exception:
            session = None

    if session:
        try:
            if cmd in ["play", "pause", "play_pause", "resume", "toggle"]:
                await session.try_toggle_play_pause_async()
                return "Playback toggled, sir."
            elif cmd in ["next", "skip", "next_track"]:
                await session.try_skip_next_async()
                return "Skipped to next track, sir."
            elif cmd in ["previous", "prev", "previous_track", "back"]:
                await session.try_skip_previous_async()
                return "Playing previous track, sir."
            elif cmd in ["stop"]:
                await session.try_stop_async()
                return "Playback stopped, sir."
        except Exception:
            pass

    if cmd in ["play", "pause", "play_pause", "resume", "toggle"]:
        if _send_browser_play_pause_background():
            return "Toggled browser video playback, sir."
        _send_hardware_media_key(VK_MEDIA_PLAY_PAUSE)
        return "Playback toggled via hardware key, sir."
    elif cmd in ["next", "skip", "next_track"]:
        _send_hardware_media_key(VK_MEDIA_NEXT_TRACK)
        return "Skipped track, sir."
    elif cmd in ["previous", "prev", "previous_track", "back"]:
        _send_hardware_media_key(VK_MEDIA_PREV_TRACK)
        return "Previous track, sir."
    elif cmd in ["mute", "unmute", "toggle_mute"]:
        _send_hardware_media_key(VK_VOLUME_MUTE)
        return "Audio mute toggled, sir."
    return "No active media stream found to control, sir."

async def get_current_media_info() -> str:
    if MediaManager is not None:
        try:
            manager = await MediaManager.request_async()
            session = manager.get_current_session()
            if not session:
                sessions = manager.get_sessions()
                if sessions and len(sessions) > 0:
                    session = sessions[0]
            if session:
                props = await session.try_get_media_properties_async()
                if props and props.title and len(props.title.strip()) > 0:
                    title = props.title.strip()
                    artist = props.artist.strip() if props.artist else ""
                    if artist:
                        return f"Currently playing '{title}' by {artist}, sir."
                    return f"Currently playing '{title}', sir."
        except Exception:
            pass

    windows = _find_browser_windows()
    for _, _, raw_title in windows:
```

```
        clean = raw_title.lower()
        if "youtube" in clean or "shorts" in clean:
            clean_title = raw_title
            for suffix in ["- Google Chrome", "- Microsoft Edge", "- Brave", "- YouTube"]:
                clean_title = clean_title.replace(suffix, "").strip()
            if clean_title.startswith("(") and ")" in clean_title:
                clean_title = clean_title.split(")", 1)[1].strip()
            if clean_title and clean_title.lower() != "youtube":
                return f"Currently playing YouTube video: '{clean_title}', sir."
    return "No active media stream or video detected, sir."

def open_website(url: str) -> str:
    clean_url = url.strip()
    if not clean_url.startswith(("http://", "https://")):
        clean_url = "https://" + clean_url
    webbrowser.open(clean_url)
    return f"Opened {clean_url}"

def open_application(app_name: str) -> str:
    query = app_name.lower().strip()
    uwp_protocols = {
        "camera": "microsoft.windows.camera:", "calculator": "calculator:",
        "calc": "calculator:", "photos": "ms-photos:", "settings": "ms-settings:",
        "spotify": "spotify:", "paint": "mspaint", "notepad": "notepad",
        "cmd": "cmd", "terminal": "wt", "explorer": "explorer", "task manager": "taskmgr"
    }
    if query in uwp_protocols:
        try:
            target = uwp_protocols[query]
            if target.endswith(":"):
                os.system(f"start {target}")
            else:
                subprocess.Popen(target, shell=True)
            return f"Launched {app_name}"
        except Exception:
            pass

    if sys.platform == "win32":
        search_dirs = [
            r"C:\ProgramData\Microsoft\Windows\Start Menu\Programs",
            os.path.expandvars(r"%APPDATA%\Microsoft\Windows\Start Menu\Programs"),
            os.path.expandvars(r"%LOCALAPPDATA%\Programs"),
        ]
        matches = []
        for directory in search_dirs:
            if not os.path.exists(directory):
                continue
            for root, _, files in os.walk(directory):
                for file in files:
                    if file.lower().endswith((".lnk", ".exe", ".url")):
                        file_name = file.lower().rsplit(".", 1)[0]
                        if query == file_name:
                            os.startfile(os.path.join(root, file))
                            return f"Launched {app_name}"
                        elif query in file_name:
                            matches.append(os.path.join(root, file))
        if matches:
            os.startfile(matches[0])
            return f"Launched {app_name}"
    return f"Could not launch {app_name}"

def close_application(app_name: str) -> str:
    query = app_name.lower().strip().replace(".exe", "")
    protected = {"explorer.exe", "svchost.exe", "system", "idle", "python.exe", "cmd.exe"}
```

```
    current_pid = os.getpid()
    killed = 0

    for proc in psutil.process_iter(['pid', 'name']):
        try:
            pid = proc.info['pid']
            pname = (proc.info['name'] or "").lower()
            if pid == current_pid or pname in protected:
                continue
            if query == pname.replace(".exe", "") or query in pname:
                proc.kill()
                killed += 1
        except Exception:
            continue
    if killed > 0:
        return f"Closed {app_name}."
    return f"Could not find any active process for {app_name}, sir."

def purge_background_tasks() -> str:
    target_signatures = [
        "crashpad_handler", "googleupdate", "microsoftedgeupdate", "edgeupdate",
        "epicwebhelper", "steamwebhelper", "discordhelper", "gamebarpresencewriter"
    ]
    protected = {"explorer.exe", "svchost.exe", "system", "python.exe"}
    initial_ram = psutil.virtual_memory().used
    killed_count = 0
    killed_names = set()

    for proc in psutil.process_iter(['pid', 'name']):
        try:
            pid = proc.info['pid']
            raw_name = proc.info['name'] or ""
            lower_name = raw_name.lower()
            if pid == os.getpid() or lower_name in protected:
                continue
            if any(sig in lower_name for sig in target_signatures):
                proc.kill()
                killed_count += 1
                killed_names.add(raw_name.replace(".exe", "").title())
        except Exception:
            continue

    freed_mb = max(0, int((initial_ram - psutil.virtual_memory().used) / (1024 * 1024)))
    if killed_count > 0:
        names_str = ", ".join(list(killed_names)[:3])
        return f"Terminated {killed_count} tasks including {names_str}, freeing {freed_mb} MB RAM, sir."
    return "Background tasks are already optimized, sir."
```

■ TERMINAL II // agent.py (Gemini 3.1 Live Neural Gateway)

```
"""
agent.py
Real-Time Gemini Live Engine with Hands-Free Media Navigation (Episode 13)
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import math
import struct
import pyaudio
import winsound
import keyboard
from dotenv import load_dotenv

from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console

from hardware_control import (
    get_brightness, set_brightness, get_volume, set_volume,
    get_battery_status, get_system_vitals, get_system_temperatures, restore_master_volume
)
from software_control import (
    open_website, open_application, close_application, purge_background_tasks,
    control_media, get_current_media_info, is_media_actively_playing,
    navigate_shorts, skip_video, seek_video, toggle_video_view
)

load_dotenv()

console = Console()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()
model_is_speaking = False
media_is_playing_state = False

def calculate_rms(data: bytes) -> float:
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

tools = [
    open_website, open_application, close_application, purge_background_tasks,
    control_media, get_current_media_info, navigate_shorts, skip_video,
    seek_video, toggle_video_view, set_volume, get_volume,
    set_brightness, get_brightness, get_battery_status,
    get_system_vitals, get_system_temperatures
]

def get_time_greeting() -> str:
    hour = datetime.datetime.now().hour
    if 5 <= hour < 12:
```

```
        return "Good morning, sir."
    elif 12 <= hour < 17:
        return "Good afternoon, sir."
    return "Good evening, sir."

async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
        except Exception:
            pass
        await asyncio.sleep(1.0)

async def send_audio(session):
    global model_is_speaking, media_is_playing_state
    mic_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_IN, input=True, frames_per_buffer=CHUNK)
    silence_frame = b'\x00' * (CHUNK * 2)

    try:
        while True:
            data = await asyncio.to_thread(mic_stream.read, CHUNK, exception_on_overflow=False)
            if model_is_speaking:
                await session.send_realtime_input(audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"})
                continue
            if media_is_playing_state:
                if keyboard.is_pressed("right ctrl"):
                    await session.send_realtime_input(audio={"data": data, "mime_type": "audio/pcm;rate=16000"})
                else:
                    await session.send_realtime_input(audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"})
            else:
                rms = calculate_rms(data)
                if rms > 450:
                    await session.send_realtime_input(audio={"data": data, "mime_type": "audio/pcm;rate=16000"})
                else:
                    await session.send_realtime_input(audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"})
    except asyncio.CancelledError:
        raise
    finally:
        mic_stream.stop_stream()
        mic_stream.close()

async def receive_audio(session, audio_queue):
    global model_is_speaking

    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        result = ""
                        if call.name == "navigate_shorts":
                            result = navigate_shorts(call.args.get("direction", "next"))
                        elif call.name == "skip_video":
                            result = skip_video(call.args.get("direction", "next"))
                        elif call.name == "seek_video":
                            result = seek_video(call.args.get("action", "forward"))
                        elif call.name == "toggle_video_view":
                            result = toggle_video_view(call.args.get("mode", "fullscreen"))
                        elif call.name == "control_media":
```

```
                            result = await control_media(call.args.get("action", "play_pause"))
                        elif call.name == "open_application":
                            result = open_application(call.args.get("app_name", ""))
                        elif call.name == "close_application":
                            result = close_application(call.args.get("app_name", ""))
                        elif call.name == "purge_background_tasks":
                            result = purge_background_tasks()

                        function_responses.append(types.FunctionResponse(name=call.name, id=call.id, response={"result": result}))
                    if function_responses:
                        await session.send_tool_response(function_responses=function_responses)

                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:
                        while not audio_queue.empty():
                            audio_queue.get_nowait()
                        model_is_speaking = False
                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        model_is_speaking = True
                        for part in model_turn.parts:
                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)
                    if server_content.turn_complete:
                        model_is_speaking = False
    finally:
        await audio_queue.put(None)

async def play_audio(audio_queue):
    speaker_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_OUT, output=True, frames_per_buffer=CHUNK)
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    finally:
        speaker_stream.stop_stream()
        speaker_stream.close()

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        speech_config=types.SpeechConfig(voice_config=types.VoiceConfig(prebuilt_voice_config=types.PrebuiltVoiceConfig(voice_name="P
uck"))),
        system_instruction=types.Content(parts=[types.Part.from_text(
            "You are JARVIS, built by Swayam Dhawale, Founder of Pulse of AI. "
            "Polite, concise Indian English accent. Always address user as 'sir'. "
            "Execute tools immediately for media navigation, YouTube shorts scrolling, volume, and app control."
        )])
    )
    while True:
        try:
            async with client.aio.live.connect(model=model_id, config=config) as session:
                audio_queue = asyncio.Queue()
                await asyncio.gather(
                    monitor_playback_state(),
                    send_audio(session),
                    receive_audio(session, audio_queue),
                    play_audio(audio_queue)
                )
        except Exception as e:
            await asyncio.sleep(1)

if __name__ == "__main__":
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    asyncio.run(main())
```